

Session 7: Testing a Return-Prediction Model

You are given a gradient boosting model that was trained to predict stock returns, and six years of data it has never seen. Your job is to find out whether it works.

Two files, both in `data/`:

File	What it is
<code>session7_model.joblib</code>	The fitted model, plus the feature list it expects
<code>session7_test.parquet</code>	2021-01 through 2026-08, 124,125 stock-months

The data

One row per stock per month: 68 months, about 1,825 stocks a month.

The timing convention is the whole point of the file, so be sure you have it:

`ret` is the return earned **over** month `t`. Every feature in that row was known at the **close of month `t-1`**, before the return started.

Nothing in a row is contemporaneous with its own return. A model built on these columns is making a real forecast, not describing the present.

Column	Meaning
<code>month</code>	Year and month, as a string like <code>2021-07</code>
<code>ticker</code> , <code>name</code>	Identifiers
<code>ret</code>	Total return over the month, including dividends and delistings
<code>target</code>	What the model predicts: <code>ret</code> 's percentile rank within the month

The fifteen features

Every feature is a **cross-sectional percentile rank within its month**, between 0 and 1. A **momentum** of 0.93 means that stock had higher momentum than 93% of the stocks in that month, not that it returned 93%. Missing values were filled at 0.5, the middle of the cross section.

Ranking matters for a reason worth understanding: a dollar threshold does not hold still over twenty-five years. The median market cap of the stocks in this universe rose about fivefold between 2001 and 2026, so a split the model learned as "market cap above four billion" would land somewhere completely different by the end of the sample. A percentile does not drift.

Price and volume, measured over or at the end of month t-1:

Feature	Definition
momentum	11-month return ending at the close of month t-2 — month t-1 is skipped
ret_1m	The return in month t-1 alone (short-term reversal)
volatility	Standard deviation of daily returns during month t-1
volatility_12m	Standard deviation of monthly returns over months t-12 through t-1
illiquidity	Amihud: average daily return per dollar of volume in month t-1
dollarvol	Average daily dollar volume during month t-1
marketcap	Market capitalization at the close of month t-1

Valuation, at the close of month t-1:

Feature	Definition
pb	Price to book
ps	Price to sales

Accounting, from the most recent 10-K **filed before the month began**:

Feature	Definition
roe	Return on equity
grossprofitability	Gross profit divided by total assets

Feature	Definition
<code>accruals</code>	$(\text{Net income} - \text{cash flow from operations}) \div \text{average total assets}$
<code>assetgrowth</code>	Change in total assets since the previous annual filing
<code>issuance</code>	Log change in shares outstanding since the previous annual filing
<code>divyield</code>	Dividend yield

The accounting data is as-originally-reported and is keyed on the SEC filing date, not the fiscal period end. A 10-K covering fiscal 2023 but filed in February 2024 first appears in the March 2024 row. No row anywhere in this file uses a filing that did not exist when the month began.

The universe

Market-cap ranks 1001 through 3000 each month — the thousand largest companies are excluded, so this is a small-cap universe roughly like the Russell 2000 — and a closing price above \$5 in the prior month. The price filter removes a median of 139 stocks a month, but over 600 in early 2009, which is worth remembering if you ever extend this backwards.

How the model was trained

`GradientBoostingRegressor` from scikit-learn, fitted on **2001-01 through 2020-12**: 240 months and 442,477 stock-months. You do not have that data, and that is deliberate — the model has never seen a single row of the file you do have.

The target is the percentile rank of return within the month, not the return itself. Ranking within the month removes the market: in a month when everything fell 9%, the model is still being asked which stocks fell least. It also gives every month equal weight in the loss, so March 2020 does not drown out the other 239 months, and it makes outliers harmless — a 400% return is simply rank 1.0.

Hyperparameters came from `GridSearchCV` with `n_estimators` fixed at 300 and a grid over `learning_rate` (0.02 to 0.30) and `max_depth` (2, 3, 5). Two details of that

search are not the defaults, and both matter:

- **The folds are expanding windows of whole months**, not **KFold**. Whole months, because stocks within a month share the market and splitting one across a fold boundary lets the model see half of a month while being graded on the other half. Expanding, because the alternative trains on 2018 to predict 2006.
- **The score is the mean monthly rank correlation**, not R^2 . R^2 on a rank target sits near zero and goes negative regularly, so it cannot tell a good model from a bad one.

The winner was **learning_rate=0.02, max_depth=2** with a cross-validated IC of 0.0385. Depth 2 beat depths 3 and 5 at every learning rate — noisy financial data rewards very simple trees.

No industry variables are in the model. An earlier version included sector dummies; they absorbed 40% of the fitted importance and the model's out-of-sample IC fell to zero, because what it had really learned was which sectors did well in the training years.

What to do

Do not refit the model. Load it and evaluate it.

1. Predict

```
import joblib, pandas as pd

bundle = joblib.load("data/session7_model.joblib")
model, features = bundle["model"], bundle["features"]

df = pd.read_parquet("data/session7_test.parquet")
df["pred"] = model.predict(df[features])
```

2. Information coefficient

For each month, the Spearman correlation between prediction and realized return. Average it across the 68 months and test whether that average is different from zero. This is the standard measure of a return forecast. A sustained monthly IC of 0.02 to 0.05 is a genuinely useful signal; treat anything above 0.10 as a reason to hunt for a bug.

3. Sort into deciles

Each month, sort the stocks on **pred** and split them into ten equal groups. Compute the equal-weighted average **ret** of each decile, then average across months.

Report the decile means and look at the shape. Is it monotone? Where does the spread come from — the top, the bottom, or both?

4. The long-short portfolio and its Sharpe ratio

Form the monthly return series of decile 10 minus decile 1. Report its mean, its standard deviation, and its annualized Sharpe ratio. The portfolio is self-financing, so do not subtract the risk-free rate from it. Do subtract it if you also evaluate decile 10 on its own, which you should.

5. Fama-French regressions

Regress the long-short monthly returns on the five Fama-French factors:

```
import pandas_datareader.data as web
ff5 = web.DataReader("F-F_Research_Data_5_Factors_2x3", "famafrch",
                    start="2020-01-01") [0] / 100.0
```

The factors are in percent, so divide by 100. They are published with a lag, so the last month or two of your panel may have no factor data — intersect the dates rather than assuming they line up.

Report the intercept, annualize it, and test it. Then report the loadings and say what they mean. If the strategy earns its return by loading on profitability and value, the honest conclusion is that the model rediscovered known factors rather than finding something new. Run it again adding the momentum factor (**F-F_Momentum_Factor**) and see whether the alpha survives.

6. Compare against something simpler

A number on its own means nothing. Run the same decile test on two single signals, sorting on the feature instead of on **pred**:

- **momentum** on its own.
- **volatility** on its own, sorted so that LOW volatility is the long side. This is the model's most heavily used feature by a wide margin.

The model gets fifteen features and three hundred trees. If it cannot beat a one-line sort on a single column, that is a finding, and you should report it as one rather than burying it.